

Train vs Validation Data

How we know a model really learned · video 1:53

This lesson answers one question: how do we know a model has really **learned**, and not just memorized? Before training we split the data, never train on one slice, the **validation set**, and measure the loss there too. Comparing the two losses tells us whether the model is learning or **overfitting**.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 9).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. A low training loss can fool us	▶ 0:08-0:26	4	☐
2. Split the data before training	▶ 0:26-0:50	3	☐
3. Two losses, and the gap between them	▶ 0:50-1:13	4	☐
4. Overfitting	▶ 1:13-1:25	4	☐
5. The test set: one final, honest score	▶ 1:25-1:32	3	☐
6. The split in code	▶ 1:32-1:39	2	☐

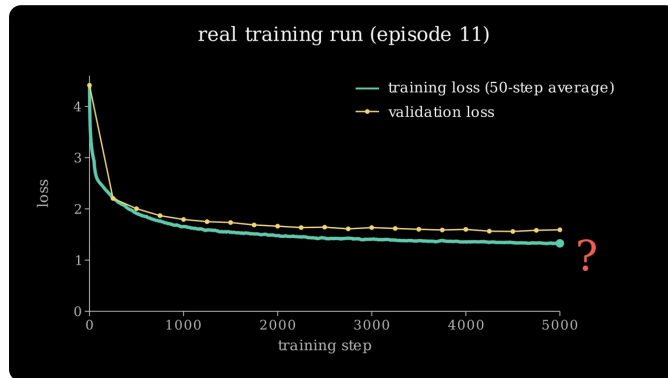
Before you start: this lesson builds on F12 (training nudges the parameters) and F13 (the training loop). All you need about the **loss**: it measures how wrong the model's predictions are, so lower is better. The two loss curves appear in episode 11 (Training: Learning from Mistakes), and this exact split in the tiny GPT of episode 12 (Build a Tiny GPT).

CONCEPT 1 OF 6

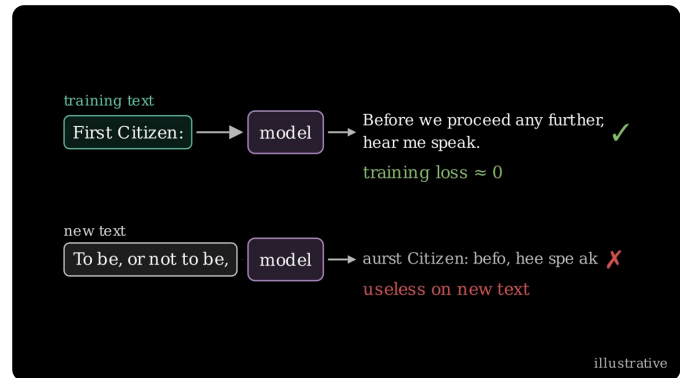
A low training loss can fool us

▶ Video 0:08–0:26

Episode 11 showed two loss curves, one for **training** (teal) and one for **validation** (yellow). Why two? Because a low training loss, on its own, can fool us. A model could simply **memorize** its training text, word for word. It would score perfectly on that text, a training loss of about 0, and be useless on anything new.



▶ 0:16 Episode 11's real run. Why two curves?



▶ 0:25 A memorizer (illustrative).

KEY IDEA

The loss on the training data shows how well the model **fits what it has seen**, not whether it has **learned**. To judge learning, measure on text it has never seen.

Check 1: A low training loss can fool us

Answer all 4 questions before looking at the answers. Pass mark: all correct.

1.1 MULTIPLE CHOICE

A student scores 100 % on a practice exam after studying the answer key of that exact exam. What does the score tell you?

- A. The student has mastered the subject
- B. Very little: they may just have memorized the answers, so test them on new questions
- C. The exam was too easy for everyone
- D. They will score 100 % on any exam

1.2 TRUE OR FALSE

"A training loss close to 0 proves that the model will do well on new text."

True ☐ False ☐

1.3 MULTIPLE CHOICE

Which measurement would reveal that the video's memorizer has not really learned?

- A. Its loss on the training text
- B. Its loss on text it was never trained on
- C. How long training took
- D. How many parameters it has

1.4 SHORT ANSWER

In one sentence: what is the difference between memorizing and learning?

.....

CONCEPT 2 OF 6

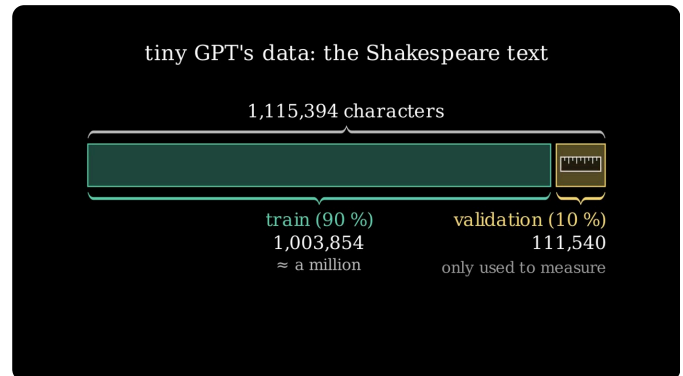
Split the data before training

▶ Video 0:26–0:50

So **before** training, we split the data. Most of it, the **training set**, is used for learning. A slice we never train on, the **validation set**, is kept aside to test with.



▶ 0:35 Most of the data is the training set; a slice we never train on is the validation set.



▶ 0:49 The tiny GPT's split: 1,003,854 characters for training, 111,540 only for measuring.

For the tiny GPT, the data is the Shakespeare text: 1,115,394 characters. The first 90 %, **1,003,854** characters (about a million), was for training. The last 10 %, **111,540** characters, was only ever used to measure.

KEY IDEA

Split first. The **training set** is for learning; the **validation set** is never trained on, only measured.

Check 2: Split the data before training

Answer all 3 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

Check the tiny GPT's split. What is 90 % of 1,115,394, rounded down to a whole character? How many characters are left for validation?

2.2 CALCULATION

A dataset has 50,000 sentences. With the same 90 / 10 split, how many are for training and how many for validation?

2.3 TRUE OR FALSE

"It is fine to train on the validation set as well, as long as you also measure on it."

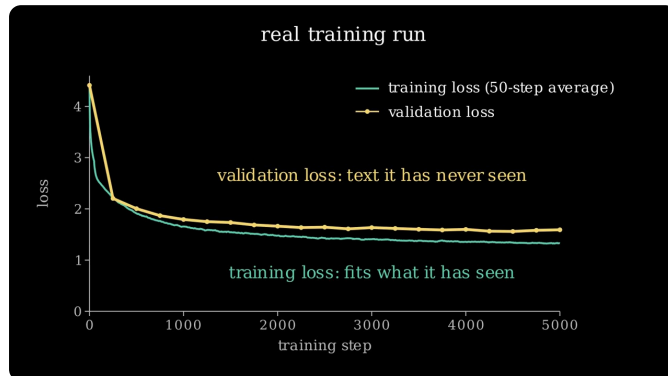
True ☐ False ☐

CONCEPT 3 OF 6

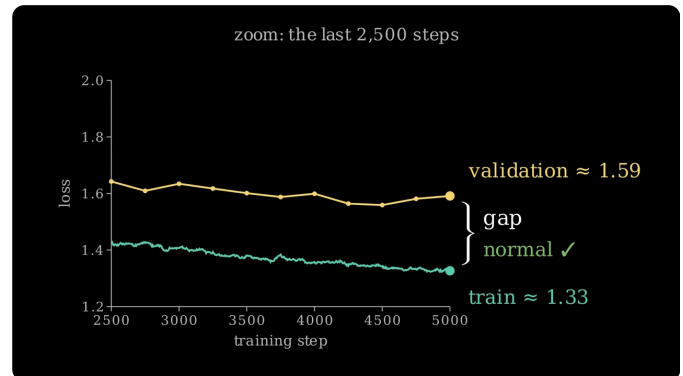
Two losses, and the gap between them

▶ Video 0:50-1:13

During training we measure the loss on **both** sets. The **training loss** says how well the model fits what it has seen. The **validation loss** says how well it does on text it has **never seen**, which is what we really care about.



▶ 1:00 Both losses, measured during the whole run. (The training curve is a 50-step average.)



▶ 1:12 The last 2,500 steps: training ends at about 1.33, validation at about 1.59.

The tiny GPT ended at about **1.33** on training text and about **1.59** on validation text. That **gap** is normal: a model usually does a little better on what it studied.

KEY IDEA

Training loss = fit to seen text. Validation loss = performance on unseen text. A small, steady gap between them is normal.

Check 3: Two losses, and the gap between them

Answer all 4 questions before looking at the answers. Pass mark: all correct.

3.1 CALCULATION

How big is the gap between the tiny GPT's final losses?

3.2 MULTIPLE CHOICE

Which number best predicts how the tiny GPT will do on a new piece of Shakespeare?

- A. The training loss, 1.33
- B. The validation loss, 1.59
- C. Their average, 1.46
- D. Neither: only a loss of 0 means anything

3.3 TRUE OR FALSE

"The validation loss is a little higher than the training loss, so the tiny GPT is broken."

True ☐ False ☐

3.4 CALCULATION

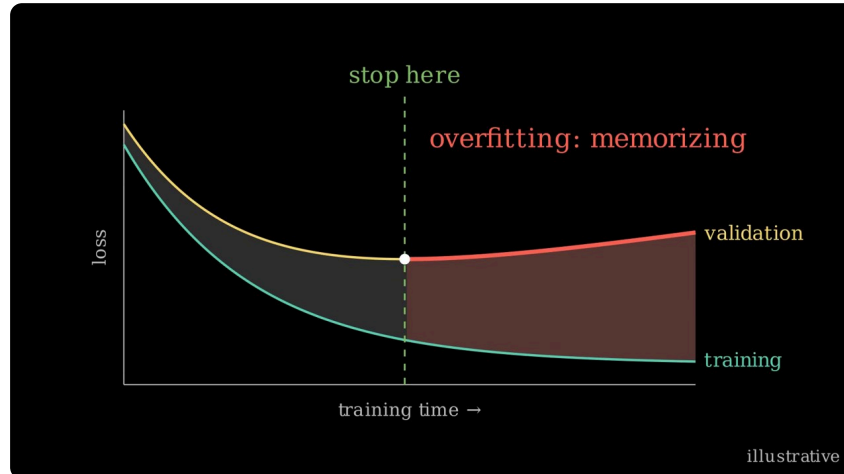
The tiny GPT's validation loss was 1.79 at step 1,000 and 1.59 at step 5,000. By how much did it fall? What does a falling validation loss tell you?

CONCEPT 4 OF 6

Overfitting

▶ Video 1:13-1:25

Watch the gap over time. If it **keeps growing** while the validation loss **stops falling**, or even **rises**, the model is **overfitting**: memorizing instead of learning. The training loss still improves, but the improvement no longer carries over to new text. That is the time to **stop**, or to find **more data**.



▶ 1:24 Overfitting (illustrative): the gap keeps growing, and the validation loss stops falling and rises. Stop at its lowest point.

KEY IDEA

Overfitting: training loss still falling, validation loss flat or rising, gap growing. Stop at the lowest validation loss, or get more data.

Check 4: Overfitting

Answer all 4 questions before looking at the answers. Pass mark: all correct.

4.1 MULTIPLE CHOICE

Which pattern shows overfitting?

- A. Both losses falling together, with a small, steady gap
- B. Training loss falling, validation loss rising
- C. Both losses flat and high from the start
- D. Validation a little above training, both still falling

4.2 CALCULATION

A run logs these losses. Step 1,000: train 2.05, val 2.10 · 2,000: 1.70, 1.85 · 3,000: 1.50, 1.78 · 4,000: 1.35, 1.81 · 5,000: 1.20, 1.90. (a) Which checkpoint would you keep? (b) What is the gap at step 1,000 and at step 5,000?

.....

.....

4.3 TRUE OR FALSE

“When a model overfits, its training loss goes up.”

True ☐ False ☐

4.4 SHORT ANSWER

Name the two remedies the video gives for overfitting.

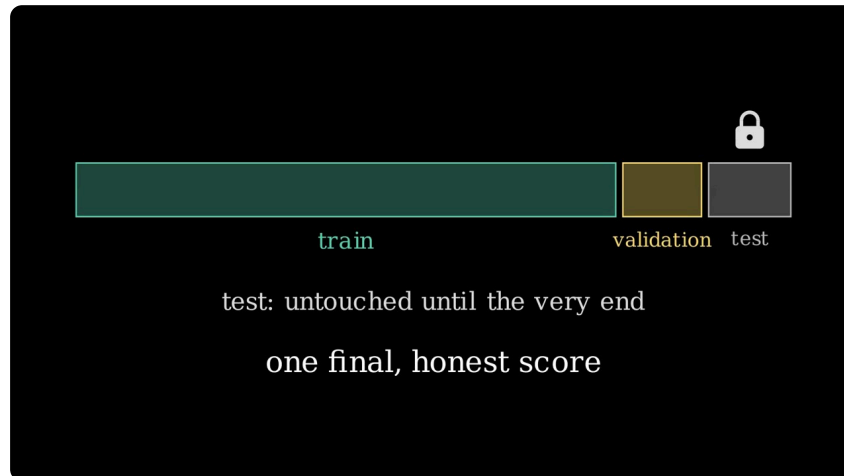
.....

CONCEPT 5 OF 6

The test set: one final, honest score

▶ Video 1:25–1:32

Careful projects keep a **third slice**, the **test set**, untouched until the very end. Why, if there is already a validation set? Because we *use* the validation loss to make decisions, such as when to stop. Every decision tunes the model a little to that slice. The test set, used once at the end, gives **one final, honest score**.



▶ 1:31 A third slice, the test set, stays locked until the very end.

KEY IDEA

Train on the **training set**, steer with the **validation set**, and score once on the **test set**, at the very end.

Check 5: The test set: one final, honest score

Answer all 3 questions before looking at the answers. Pass mark: all correct.

5.1 MULTIPLE CHOICE

Which set do you use to decide when to stop training?

- A. The training set
- B. The validation set
- C. The test set
- D. All three, averaged

5.2 TRUE OR FALSE

"It is fine to check the test score after every training step, to see how things are going."

True ☐ False ☐

5.3 PUT IN ORDER

Put in order: *report the test score* · *split the data three ways* · *keep the checkpoint with the lowest validation loss* · *train while watching the validation loss*.

CONCEPT 6 OF 6

The split in code

▶ Video 1:32-1:39

In the tiny GPT, the split is two lines. `split` is the position 90 % of the way through the data; `data[:split]` is everything before it (training), `data[split:]` everything from it on (validation). So the validation set is the **last** 10 % of the text, one continuous piece.

```
1 data = torch.tensor(encode(text))
2 split = int(0.9 * len(data))
3 train_data = data[:split]      # 1,003,854 characters
4 val_data = data[split:]        # 111,540 characters
```

▶ 1:38 The split from the tiny GPT: the first 90 % for training, the rest for validation.

```
data = torch.tensor(encode(text))
split = int(0.9 * len(data))
train_data = data[:split]      # 1,003,854 characters
val_data = data[split:]        # 111,540 characters
```

KEY IDEA

`data[:split]` and `data[split:]`: every character lands in exactly one of the two sets, and none is shared.

Check 6: The split in code

Answer all 2 questions before looking at the answers. Pass mark: all correct.

6.1 MULTIPLE CHOICE

Why is `int(...)` needed in `split = int(0.9 * len(data))`?

- A. To round 0.9 up to 1
- B. A slice position must be a whole number, and $0.9 \times 1,115,394 = 1,003,854.6$ is not
- C. To shuffle the data first
- D. To turn characters into numbers

6.2 HANDS-ON CODE

Try it yourself. The code below does the same split on the first line of the Shakespeare text, one token per character. (a) What does it print? (b) Change it to a three-way split: the first 80 % for training, the next 10 % for validation, the last 10 % for testing. How long is each part? (c) With the same 80 / 10 / 10 code, how many characters would each part get from the full 1,115,394?

```
text = "First Citizen: Before we proceed any further, hear me speak."
data = list(text)                # one token per character, like the tiny GPT
split = int(0.9 * len(data))
train_data = data[:split]
val_data = data[split:]
print(len(data), split, len(val_data))
print("".join(val_data))
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. A low training loss can fool us · rewatch ▶ 0:08-0:26

- 1.1 B.** The same trap as a low training loss: a perfect score on seen material can be pure memory.
- 1.2 False.** A memorizer also gets a training loss near 0 and is useless on new text.
- 1.3 B.** On unseen text the memorizer's answers are gibberish, so that loss would be high.
- 1.4 Memorizing reproduces the seen text; learning finds patterns that also work on new text.** Only learning makes the model useful on text it has never seen, which is what we want.

2. Split the data before training · rewatch ▶ 0:26-0:50

- 2.1 1,003,854 and 111,540.** $0.9 \times 1,115,394 = 1,003,854.6$, rounded down to 1,003,854; then $1,115,394 - 1,003,854 = 111,540$.
- 2.2 45,000 and 5,000.** $0.9 \times 50,000 = 45,000$; the remaining 10 % is 5,000.
- 2.3 False.** Then it is no longer text the model has never seen, and its loss can be fooled by memorizing, just like the training loss.

3. Two losses, and the gap between them · rewatch ▶ 0:50-1:13

- 3.1 0.26.** $1.59 - 1.33 = 0.26$.
- 3.2 B.** The validation loss is measured on text the model never trained on, like the new piece.
- 3.3 False.** A small gap is normal: a model usually does a little better on what it studied.
- 3.4 0.20; the model was still getting better on unseen text.** $1.79 - 1.59 = 0.20$. Memorizing cannot lower the loss on unseen text, so the improvement is real learning.

4. Overfitting · rewatch ▶ 1:13-1:25

- 4.1 B.** The model keeps fitting its training text better while getting worse on unseen text: memorizing, not learning.
- 4.2 (a) Step 3,000. (b) 0.05 and 0.70.** The validation loss is lowest (1.78) at step 3,000 and rises after it, while the gap grows from $2.10 - 2.05 = 0.05$ to $1.90 - 1.20 = 0.70$: overfitting.
- 4.3 False.** The training loss keeps going down. It is the validation loss that stalls or rises.
- 4.4 Stop training (at the lowest validation loss), or find more data.** Stopping keeps the best model; more data makes memorizing harder than learning.

5. The test set: one final, honest score · rewatch ▶ 1:25-1:32

- 5.1 B.** Stopping is a decision made during the project; the test set must stay untouched until the end.
- 5.2 False.** Then the test set steers your decisions like a validation set, and its score is no longer an honest final check.
- 5.3 split → train and watch → keep the best checkpoint → report the test score.** The test set is touched only once, at the very end.

6. The split in code · rewatch ► 1:32-1:39

6.1 B. `int` drops the .6, giving 1,003,854. Turning characters into numbers is `encode`'s job.

6.2 (a) 60 54 6, then speak. · (b) 48, 6, 6 · (c) 892,315 / 111,539 / 111,540 (a) 60 characters, $0.9 \times 60 = 54$, so the last 6 characters, *speak.*, are the validation set. (b) `a, b = int(0.8 * len(data)), int(0.9 * len(data))`, then `train_data, val_data, test_data = data[:a], data[a:b], data[b:]`. Lengths 48, 6 and 6. (c) $a = \text{int}(0.8 \times 1,115,394) = 892,315$ and $b = 1,003,854$, so 892,315 for training, $1,003,854 - 892,315 = 111,539$ for validation, and 111,540 for testing. Rounding down makes the middle part one character shorter; all three still add up to 1,115,394.